N046 AI 서비스 인 프로덕션_학습서

Chapter01

Google Cloud에 처음 배포하기

지금까지 메시지에 따라 모델이 갈 길을 고르게 하고, 필요한 도구를 불러 결과를 답에 넣고, 이전 대화를 이어 붙이는 에이전트를 만들어 보았습니다. 이 교안에서는 이런 요소들을 FastAPI 하나로 묶은 AI 친구 에이전트 서비스를 예로 들어, 에이전트를 실제 서비스로 내놓는 과정을 실습합니다. 사용자가 성격과 말투를 정한 캐릭터가 대화를 기억하고, 날씨나 공휴일을 찾아 답하고, 부탁한 약속을 저장했다가 먼저 말을 거는 서비스입니다. 서비스는 실습용 템플릿 저장소에 준비되어 있고, 이 교안에서는 줄여서 에이전트 서비스라고 부르겠습니다.

이 에이전트 서비스를 동료에게 보여 주면 "나도 써 보게 주소 좀 줘"라는 말이 돌아옵니다. 그런데 줄 주소가 없습니다. 서비스는 로컬 환경(내 컴퓨터) 안에서만 돌아가고, 컴퓨터를 끄면 함께 멈춥니다. 주소를 만드는 것만이 문제도 아닙니다. 에이전트 서비스는 LLM 제공사(OpenAI, Google처럼 LLM API를 제공하는 회사)가 발급한 API 키, 곧 LLM API 키를 숨겨야 하고, 메시지 하나에 LLM API를 여러 번 호출하느라 답이 늦어지기도 하며, 메시지가 들어올 때마다 LLM 제공사에 돈을 냅니다. 오늘은 Google Cloud를 처음부터 준비하고, 에이전트 서비스를 Cloud Run에 올려 첫 공개 주소를 만듭니다. 배포하기 전에 로컬 환경에서 먼저 실행해 보고, LLM API 키를 Secret Manager에 두는 데까지가 오늘의 범위이고, 응답 시간과 비용을 지키는 설정은 다음 실습에서 이어 갑니다.

주관식 퀴즈

Q. [시작 질문] Vercel로 배포해 본 서비스를 떠올려 보세요. 배포 버튼을 누른 뒤 공개 주소가 생기기까지 Vercel 안에서 어떤 일이 일어났다고 생각하나요? 아는 만큼, 짐작하는 만큼 적어 보세요.

Chapter02

Vercel로 프론트엔드(frontend, 사용자가 브라우저에서 보는 화면)를 배포할 때는 저장소를 연결하고 버튼을 누르면 주소가 생겼습니다. 프론트엔드만 있는 서비스라면 에이전트 서비스도 그렇게 올리면 됩니다. 그런데 에이전트 서비스는 LLM API에 질문을 보내고 답을 기다리는 서버이고, 이런 서버는 Vercel이 정해 둔 실행 시간과 실행 조건 안에 들어가지 않는

경우가 생깁니다. 아래 그림은 두 방식에서 내가 올리는 것과 플랫폼이 준비하는 것을 나란히 놓은 것입니다.



그림: Vercel과 서버리스 컨테이너 비교. 내가 올리는 것과 플랫폼이 준비하는 것이 다르다

서비스가 실행되려면 먼저 코드가 있어야 하고, 코드에는 프론트엔드와 백엔드(backend, 요청을 받아 처리하고 결과를 돌려주는 서버 쪽 코드)가 함께 들어갑니다. 코드를 실행하는 Python 버전을 런타임(runtime), 코드가 가져다 쓰는 라이브러리를 의존성(dependencies)이라고 부르며, 문서 변환 프로그램처럼 운영체제에 직접 설치하는 시스템 패키지가 필요할 때도 있습니다. Vercel은 코드만 받고 나머지는 Vercel이 준비한 것을 쓰므로, 고를 수 있는 범위가 Vercel이 허용한 선택지 안으로 정해집니다. 서버리스 컨테이너는 코드와 런타임, 의존성, 시스템 패키지를 컨테이너 이미지 하나로 묶어 받고, 플랫폼은 그 이미지를 그대로 실행합니다.

두 방식 모두 서버리스입니다. 요청량에 맞춰 서버를 켜고 끄고 개수를 늘리고 줄이는 오토스케일링(autoscaling)을 플랫폼이 맡고, 요청이 없으면 실행 중인 서버가 0개로 줄어 실행 요금이 나가지 않습니다. 서버를 0개까지 줄이는 동작은 스케일 투 제로(scale to zero)라고 부릅니다. 차이는 서버 관리가 아니라 런타임과 의존성을 누가 정하느냐에 있습니다.

AI 서비스에 이 차이가 중요한 이유는 시간과 도구입니다. 요청 하나를 처리할 수 있는 최대 시간을 요청 타임아웃(request timeout)이라고 합니다. Vercel 무료 플랜의 함수는 요청 타임아웃이 최대 5분이고, Cloud Run은 기본 5분에서 최대 60분까지 늘릴 수 있습니다(2026년 10월 기준). Cloud Run 콘솔에서는 이 설정이 "요청 제한 시간"으로 표시됩니다. 여러 문서를 읽고 정리하는 에이전트 작업은 5분을 넘기기 쉽고, 플랫폼이 준비해 두지 않은 시스템 패키지가 필요할 때도 많습니다. 실무에서는 프론트엔드는 Vercel에 두고 백엔드만 컨테이너로 올리는 구성도 흔한데, 에이전트 서비스는 프론트엔드가 FastAPI가 돌려주는 페이지 한 장뿐이라 Cloud Run 서비스 하나로 묶습니다. 대신 런타임과 의존성을 내가 정하므로 이미지에 빠진 것이 있으면 원인도 내가 찾아야 합니다.

이 과정에서 Cloud Run을 쓰는 이유

이 과정에서 Cloud Run을 쓰는 이유는 Vercel보다 낫기 때문이 아니라, 배포의 각 단계를 직접 보고 다룰 수 있고, 여기서 익힌 개념이 다른 클라우드와 회사 환경에서도 그대로 쓰이기 때문입니다. Vercel은 빌드부터 버전 관리, 서버 개수 조절까지 알아서 처리해 주는 대신 그 과정을 보여 주지 않습니다. Cloud Run에서는 컨테이너 이미지, 리비전, 인스턴스, 트래픽, 서비스 계정, 로그를 하나씩 직접 만지게 되고, 이 개념들은 AWS, Azure 같은 다른 클라우드와 회사가 운영하는 서버 환경에도 거의 같은 이름으로 있습니다.

직접 눌러 보며 확인하기

아래 화면의 네 탭으로 지금까지의 내용을 확인합니다. ② 탭에 나오는 인스턴스는 섹션 4에서 다시 설명합니다. ① 무엇을 올리나 탭에서는 두 방식이 준비하는 칸을 비교하고, ② 인스턴스는 언제 늘고 줄까 탭에서는 요청을 보내며 인스턴스가 0개에서 켜지고 최대 개수에서 멈추는 모습을 봅니다. 인스턴스 하나가 동시에 처리하는 요청 수를 동시성(concurrency)이라고 하는데, 이 화면은 동시성을 3으로 줄여 두었습니다. ③ 오래 걸리는 요청 탭에서는 작업 길이에 따라 두 플랫폼에서 끝까지 처리되는지, ④ 내 서비스는 어디에? 탭에서는 에이전트 서비스에 맞는 방식이 무엇인지 확인합니다.

Chapter03

Google Cloud가 하는 일

Google Cloud는 Google이 운영하는 데이터센터의 컴퓨터, 저장 공간, 네트워크를 필요한 만큼 빌려 쓰고 쓴 만큼 요금을 내는 서비스입니다. AWS, Microsoft Azure와 같은 종류이고, 이런 서비스를 묶어 클라우드라고 부릅니다. 직접 서버를 사서 설치하지 않아도 주소 하나를 가진 서비스를 몇 분 만에 띄울 수 있고, 쓰지 않을 때는 요금이 거의 나가지 않습니다.

Google Cloud에는 수백 가지 서비스가 있지만 이 과정에서 쓰는 것은 몇 개뿐입니다. 이름은 처음 보면 낯설지만 각각 하는 일은 하나씩입니다.

서비스	하는 일
Cloud Run	컨테이너 이미지를 받아 실행하고 공개 주소를 붙인다
Cloud Build	코드를 받아 컨테이너 이미지를 만든다
Artifact Registry	만들어진 이미지를 보관한다
Secret Manager	시크릿(secret)을 보관한다. 시크릿은 API 키처럼 밖에 드러나면 안 되는 값이고, 콘솔에서는 "보안 비밀"로 표시된다
IAM	누가 무엇을 할 수 있는지 권한을 정한다
Cloud Logging	서비스가 남긴 로그를 모아 보여 준다



그림: Google 계정 안에 결제 계정, 그 안에 프로젝트가 있고, 프로젝트 안에서 API를 켜고 리전에 서비스를 만든다

그림을 바깥에서 안쪽으로 읽습니다. 로그인하는 사람은 Google 계정이고, 요금은 카드가 연결된 결제 계정으로 나갑니다. 실제로 서비스와 이미지와 시크릿을 만드는 곳은 프로젝트이며, 프로젝트마다 결제 계정이 하나씩 연결됩니다. 프로젝트를 지우면 그 안의 것이 함께 지워지므로, 이 과정의 실습은 프로젝트 하나에 모아 두면 끝난 뒤 정리하기도 쉽습니다.

프로젝트에는 이름이 세 개 있습니다. 화면에 보이는 프로젝트 이름은 언제든 바꿀 수 있지만, 프로젝트 ID는 전 세계에서 하나뿐이고 만든 뒤에는 바꿀 수 없습니다. 명령에 들어가는 것은 이 프로젝트 ID입니다. 프로젝트 번호는 Google Cloud가 자동으로 붙이는 숫자로, 권한 설정에서 가끔 쓰입니다.

서비스를 쓰려면 프로젝트에서 그 서비스의 API를 먼저 켜야 합니다. 켜지 않은 서비스는 명령을 보내도 거절됩니다. 서비스가 실제로 실행되는 데이터센터 위치는 리전이라고 부르며, 이 과정은 모든 실습을 미국 중부 리전 `us-central1` 에 모읍니다. Cloud Run 요금이 가장 낮은 등급의 리전이고, 실습 결과를 한곳에서 찾을 수 있어서입니다. 오른쪽의 세 가지 방법은 모두 같은 프로젝트를 다룹니다. 오늘은 결과 확인과 결제, 시크릿 입력은 콘솔에서 하고, 나머지는 gcloud CLI 명령으로 진행합니다.

계정과 무료 체험

처음 가입하면 90일 동안 쓸 수 있는 300달러 크레딧을 받습니다(2026년 10월 기준). 체험 기간에는 요금이 청구되지 않고, 기간이 끝나거나 크레딧을 다 써도 직접 유료로 전환하지 않으면 자동으로 결제되지 않습니다. 대신 그때는 만들어 둔 서비스가 멈추고, 30일 안에

유료로 전환하지 않으면 지워집니다. 이전에 Google Cloud, Google Maps Platform, Firebase에 돈을 낸 적이 있는 계정이나 이미 무료 체험을 쓴 계정은 받을 수 없으니, 그런 경우에는 강사에게 먼저 알립니다.

가입할 때 카드 정보를 넣으면 본인 확인을 위해 0~1달러가 임시로 승인되는데, 실제 결제가 아니라 잠시 잡혔다가 풀리는 금액입니다. 회사나 학교 계정은 관리자가 Google Cloud 가입을 막아 둔 경우가 많아 개인 Google 계정을 권합니다. 결제 정보는 본인이 직접 넣는 단계라서 명령이나 코딩 에이전트로 하지 않습니다.

직접 확인할 곳: 무료 체험 시작

1. 브라우저에서 https://cloud.google.com/free 를 열고 무료로 시작하기(Get started for free)를 누릅니다.
2. 쓸 Google 계정으로 로그인합니다.
3. 국가를 대한민국으로 고르고 서비스 약관에 동의합니다.
4. 결제 정보 화면에서 계정 유형과 카드 정보를 넣고 무료 체험 시작을 누릅니다.
5. Google Cloud 콘솔(https://console.cloud.google.com)로 이동하면 완료입니다. 왼쪽 위 메뉴(☰) > 결제(Billing) > 개요에서 남은 크레딧과 남은 기간을 확인합니다.

> 📷 **캡처 필요** `cap_D64_01_free_trial_credit.png`
>
> 화면: 콘솔 > 결제 > 개요
>
> 보여 줄 것: 무료 체험 크레딧 잔액과 남은 기간이 보이는 카드 (카드 번호, 결제 계정 ID는 가린다)

프로젝트 만들기

가입이 끝나면 `My First Project` 라는 프로젝트가 자동으로 생기기도 하지만, 이 과정용 프로젝트를 새로 만들어 이름을 알아보기 쉽게 둡니다. 만들 때 이름 아래에 보이는 프로젝트 ID를 꼭 확인하고 적어 둡니다. 이름을 입력하면 ID가 자동으로 정해지는데, 이미 누가 쓰는 ID면 뒤에 숫자가 붙습니다.

직접 확인할 곳: 새 프로젝트 만들기

1. 콘솔 맨 위 가운데의 프로젝트 선택 버튼(현재 프로젝트 이름이 보이는 곳)을 누르고, 열린 창 오른쪽 위의 새 프로젝트를 누릅니다.
2. 프로젝트 이름에 `friend-agent-course` 처럼 알아보기 쉬운 이름을 넣습니다.
3. 이름 아래에 보이는 프로젝트 ID를 확인합니다. 바꾸고 싶으면 수정을 눌러 소문자, 숫자, 하이픈으로 6~30자를 넣습니다. 만든 뒤에는 바꿀 수 없습니다.

4. 결제 계정이 무료 체험 결제 계정으로 선택되어 있는지 확인하고 만들기를 누릅니다.
5. 알림이 뜨면 프로젝트 선택을 누르고, 콘솔 맨 위에 새 프로젝트 이름이 보이는지 확인합니다. 대시보드의 프로젝트 정보 카드에서 이름, ID, 번호를 함께 볼 수 있습니다.

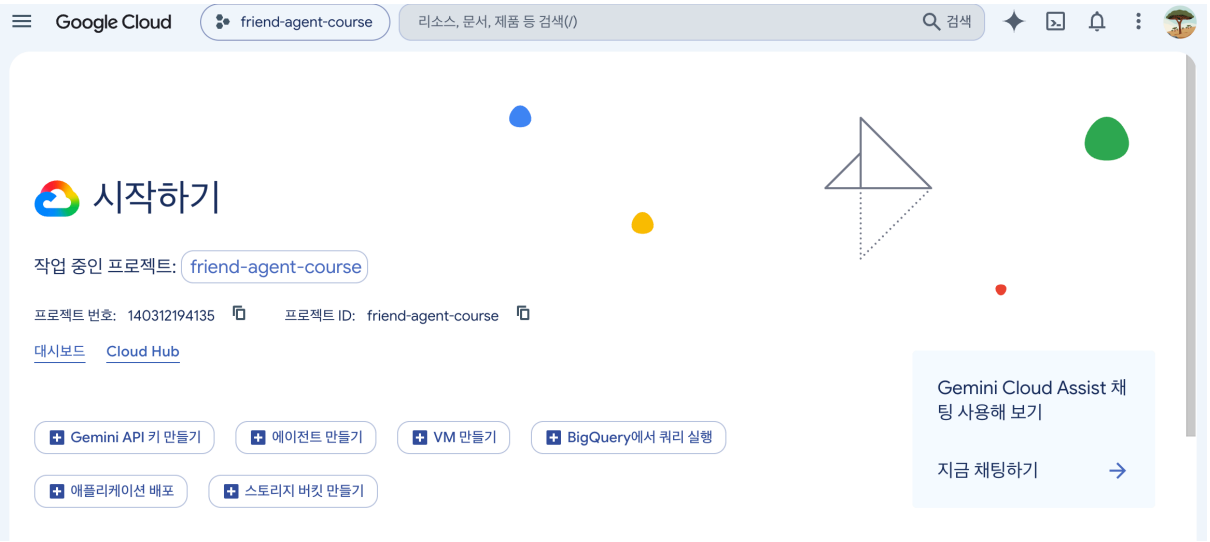


그림: 콘솔 > 대시보드 > 프로젝트 정보 카드: 프로젝트 이름, 프로젝트 ID, 프로젝트 번호가 함께 보이는 모습

콘솔에서 화면 찾기

콘솔에는 메뉴가 수백 개 있어서, 교안에 "Cloud Run > hello > 리비전 탭"이라고 적혀 있어도 처음에는 어디서 시작할지 찾기 어렵습니다. 화면을 여는 방법은 세 가지이고, 어느 방법으로 들어가든 먼저 맨 위 프로젝트 선택 버튼에 실습 프로젝트 이름이 보이는지 확인합니다. 다른 프로젝트가 선택되어 있으면 서비스 목록이 비어 보여서 배포가 안 된 것으로 착각하기 쉽습니다. 계정 언어 설정에 따라 메뉴가 영어로 보일 수 있어, 이 교안은 메뉴 이름을 괄호에 영어로 함께 적습니다.

1. 검색창: 콘솔 맨 위 가운데의 검색창에 `Cloud Run`, `Secret Manager` 처럼 서비스 이름을 입력하고, 결과 목록 위쪽의 제품 및 페이지(Products & pages) 묶음에서 고릅니다. 가장 빠른 방법입니다.
2. 탐색 메뉴: 왼쪽 위 ☰ 버튼을 누르면 서비스 목록이 열립니다. 이름 위에 마우스를 올리면 나오는 핀 아이콘을 누르면 그 서비스가 목록 맨 위에 고정됩니다. 오늘 쓰는 Cloud Run, Cloud Build, Artifact Registry, Secret Manager 네 개를 고정해 둡니다.
3. 바로가기 주소: 아래 표의 주소를 브라우저 주소창에 붙여 넣으면 그 화면이 바로 열립니다. `<프로젝트 ID>` 를 내 프로젝트 ID로 바꾸면 프로젝트 선택까지 함께 됩니다.

화면	바로가기 주소
Cloud Run 서비스 목록	`https://console.cloud.google.com/run?project=<프로젝트 ID>`
Cloud Build 빌드 기록	`https://console.cloud.google.com/cloud-build/builds?project=<프로젝트 ID>`
Artifact Registry 저장소 목록	`https://console.cloud.google.com/artifacts?project=<프로젝트 ID>`
Secret Manager 시크릿 목록	`https://console.cloud.google.com/security/secret-manager?project=<프로젝트 ID>`
사용 설정된 API 목록	`https://console.cloud.google.com/apis/dashboard?project=<프로젝트 ID>`
결제 개요	`https://console.cloud.google.com/billing`

Cloud Run 화면은 두 단계로 들어갑니다. 서비스 목록에서 서비스 이름(파란 글자)을 누르면 서비스 상세 화면이 열리고, 상세 화면 위쪽에 측정항목(Metrics), 로그(Logs), 리비전(Revisions) 같은 탭이 가로로 늘어서 있습니다. 브라우저 창이 좁으면 일부 탭이 오른쪽 화살표 뒤로 숨으니 창을 넓히거나 화살표를 누릅니다. 이 교안에서 "Cloud Run > `friend-agent` > 로그 탭"이라고 적으면 서비스 목록, 서비스 이름, 탭 순서로 누른다는 뜻입니다.

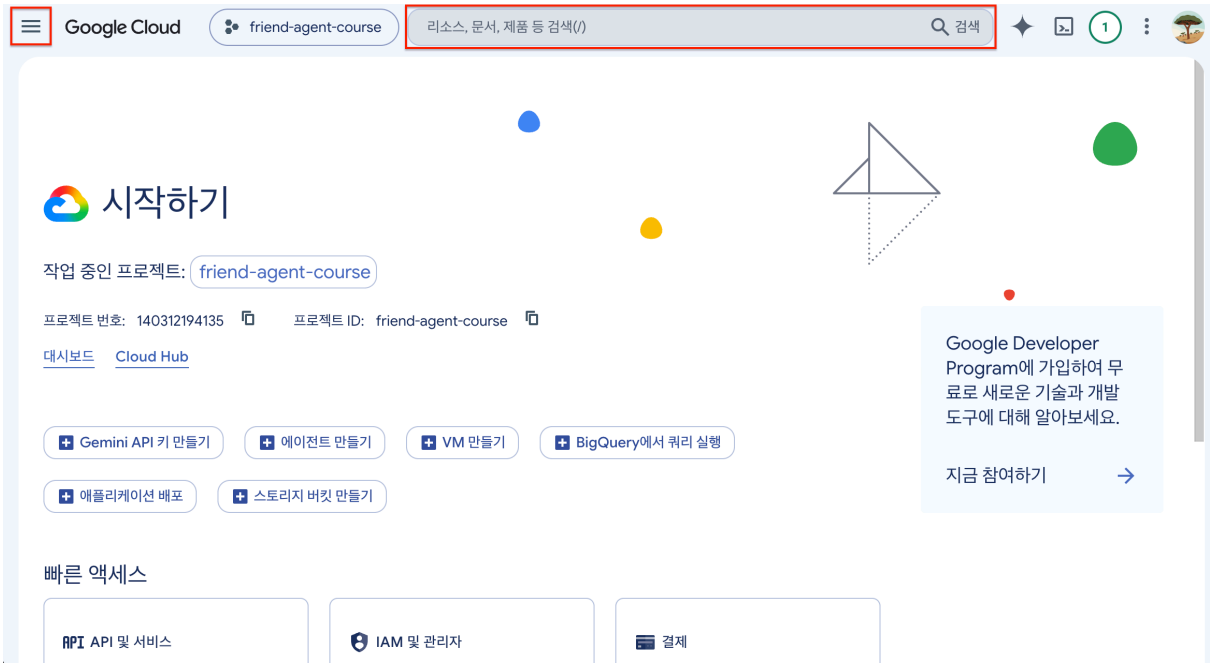


그림: 콘솔 맨 위 검색창과 왼쪽 ☰ 메뉴

gcloud CLI 설치

gcloud CLI는 콘솔에서 누르던 일을 터미널 명령으로 하게 해 주는 프로그램입니다. 이 과정에서 코딩 에이전트가 실행하는 배포 명령도 모두 로컬 환경에 설치된 gcloud를 거치므로, 에이전트를 쓰더라도 설치와 로그인은 먼저 해 두어야 합니다. 운영체제에 맞는 방법 하나만 따라 합니다.

macOS. Homebrew를 쓰고 있다면 명령 한 줄로 설치합니다. Homebrew가 없으면 공식 설치 페이지의 압축 파일로 설치합니다. 압축 파일은 칩 종류에 따라 다른데, 터미널에서 `uname -m` 을 실행해 `arm64` 가 나오면 Apple silicon용(`google-cloud-cli-darwin-arm.tar.gz`), `x86_64` 가 나오면 Intel용(`google-cloud-cli-darwin-x86_64.tar.gz`)을 받습니다.

직접 실행할 명령: macOS에 gcloud 설치

```
# 방법 1. Homebrew가 있을 때
brew install --cask gcloud-cli

# 방법 2. 공식 압축 파일 (설치 페이지에서 내 칩에 맞는 파일을 홈 폴더에 받은 뒤)
cd ~
tar -xf google-cloud-cli-darwin-arm.tar.gz     # Intel 칩이면 파일 이름이 x86_64
./google-cloud-sdk/install.sh                   # 질문에는 모두 Y. 끝나면 터미널을 새로 연다
```

Windows. 공식 설치 페이지에서 `GoogleCloudSDKInstaller.exe` 를 받아 실행합니다. 설치 옵션에서 함께 설치되는 Python은 그대로 두고, 마지막 화면의 셸 시작 옵션은 끈 채 마칩니다. 설치가 끝나면 PowerShell 창을 새로 엽니다. 설치 파일을 PowerShell에서 바로 받아 실행할 수도 있습니다.

직접 실행할 명령: Windows에 gcloud 설치 (PowerShell)

```
(New-Object Net.WebClient).DownloadFile("https://dl.google.com/dl/cloudsdk/channels/rapid/GoogleCloudSDKInstaller.exe", "$env:Temp\GoogleCloudSDKInstaller.exe")
& $env:Temp\GoogleCloudSDKInstaller.exe
```

설치가 끝나면 새 터미널에서 버전을 확인합니다. `command not found` 나 "인식되지 않는 명령"이 나오면 대부분 터미널을 새로 열지 않아서입니다. 새로 열어도 같으면 설치 스크립트가 경로(PATH)를 추가하는 질문에 Y로 답했는지 확인합니다.

직접 실행할 명령: 설치 확인

```
gcloud --version
```

코딩 에이전트에게 맡길 때: 설치 확인

```
내 컴퓨터에 gcloud CLI가 설치되어 있는지 확인하고 버전을 알려 줘.
설치되어 있지 않으면 직접 설치하지 말고, 내 운영체제와 칩 종류를 확인해서 맞는 설치 방법을 알려 줘.
```

로그인과 기본 설정

`gcloud init` 은 로그인, 프로젝트 선택을 한 번에 처리합니다. 실행하면 브라우저가 열리고, 앞에서 가입한 계정을 골라 허용을 누르면 터미널로 돌아와 프로젝트 목록을 번호로 보여 줍니다. 방금 만든 프로젝트의 번호를 입력합니다. 기본 Compute Engine 지역(zone)을 묻는 질문이 나오면 이 과정에서는 쓰지 않으므로 건너뛰어도 됩니다.

로그인은 브라우저에서 내가 직접 계정을 골라야 하는 단계라서 코딩 에이전트가 대신할 수 없습니다. 에이전트 대화창에서 명령을 직접 실행하는 기능이 있으면 그 기능으로, 없으면 별도 터미널 창에서 직접 실행합니다. 로그인이 끝나면 앞으로 쓸 기본 리전도 정해 둡니다.

직접 실행할 명령: 로그인과 기본 설정

```
gcloud init

# init 에서 프로젝트를 고르지 못했거나 바꾸고 싶을 때
gcloud config set project <프로젝트 ID>

# Cloud Run 명령의 기본 리전
gcloud config set run/region us-central1

# 확인: 로그인된 계정(* 표시), 현재 프로젝트, 기본 리전
gcloud auth list
gcloud config list
```

`gcloud auth list` 실행한 결과

```
Credentialed Accounts
ACTIVE  ACCOUNT
*       <로그인한 계정 정보>

To set the active account, run:
    $ gcloud config set account `ACCOUNT`
```

`gcloud config list` 실행한 결과

```
[core]
account = <로그인한 계정 정보>
disable_usage_reporting = False
project = <프로젝트 ID>
```

```
[run]
region = us-central1
```

오늘 쓸 API 켜기

마지막으로 오늘 쓸 네 서비스의 API를 켭니다. 프로젝트에 결제 계정이 연결되어 있어야 켜지고, 결제 계정이 없다는 오류가 나오면 앞 단계의 프로젝트 만들기에서 결제 계정을 골랐는지 확인합니다. 한 번 켜 두면 이 프로젝트에서는 다시 켤 필요가 없습니다.

직접 실행할 명령: API 사용 설정

```
gcloud services enable run.googleapis.com cloudbuild.googleapis.com artifactregistry.g
oogleapis.com secretmanager.googleapis.com

# 켜진 API 목록에서 네 서비스를 찾는다
gcloud services list --enabled
```

코딩 에이전트에게 맡길 때: API 사용 설정

```
지금 gcloud에 설정된 프로젝트에서 Cloud Run, Cloud Build, Artifact Registry, Secret Manage
r API를 사용 설정해 줘.
실행하기 전에 대상 프로젝트 ID를 먼저 보여 주고, 끝나면 네 API가 켜져 있는지 목록으로 확인해 줘.
```

에이전트가 보여 준 프로젝트 ID가 방금 만든 프로젝트와 같은지 확인하고 승인합니다. 콘솔에서는 ☰ 메뉴 > API 및 서비스(APIs & Services) > 사용 설정된 API 및 서비스(Enabled APIs & services)를 열면 같은 목록이 표로 나옵니다. 표 위쪽의 필터 칸에 `Cloud Run` 처럼 이름을 입력하면 한 줄씩 찾기 쉽습니다.

자주 막히는 곳

처음 설치할 때 막히는 지점은 대부분 정해져 있습니다. 증상을 찾아 그 줄의 확인할 것부터 봅니다. 혼자 풀리지 않으면 화면이나 터미널 메시지를 그대로 캡처해 강사에게 보여 주세요. 메시지를 옮겨 적다 보면 중요한 단어가 빠지기 쉽습니다.

증상	확인할 것
무료 체험 가입에서 카드가 거절된다	해외 결제가 되는 카드인지, 카드사 앱에서 해외 결제를 막아 두지 않았는지 확인한다. 다른 카드로 다시 시도한다
무료 체험 크레딧이 보이지 않는다	예전에 Google Cloud, Firebase, Maps에 돈을 낸 적이 있는 계정이면 받을 수 없다. 강사에게 알리고 다른 개인 계정을 쓴다

증상	확인할 것
회사·학교 계정으로 콘솔에 들어갈 수 없다	관리자가 막아 둔 경우가 많다. 개인 Google 계정으로 바꾼다
`gcloud` 를 찾을 수 없다는 메시지	터미널을 새로 연다. macOS 압축 파일 설치라면 `install.sh` 의 PATH 질문에 Y로 답했는지, Windows라면 설치 뒤 PowerShell 창을 새로 열었는지 본다
PowerShell에서 "스크립트를 실행할 수 없으므로" 오류	명령 프롬프트(cmd)에서 같은 명령을 실행해 본다. 계속 쓰려면 실행 정책을 바꿔야 하므로 강사와 함께 확인한다
`gcloud init` 에서 브라우저가 열리지 않는다	터미널에 나온 주소를 복사해 브라우저에 직접 붙여 넣는다
회사 컴퓨터라 설치가 막힌다	아래 Cloud Shell을 쓴다
API를 켜는데 결제 계정 오류가 난다	프로젝트에 무료 체험 결제 계정이 연결되어 있는지 콘솔의 결제 메뉴에서 확인한다

설치가 막힐 때: Cloud Shell

회사 컴퓨터라 프로그램을 설치할 수 없거나 설치가 계속 실패하면 Cloud Shell을 씁니다. 콘솔 오른쪽 위의 터미널 모양 아이콘(Cloud Shell 활성화)을 누르면 브라우저 아래쪽에 터미널이 열리는데, gcloud가 이미 설치되어 있고 콘솔에 로그인한 계정으로 바로 연결됩니다. `gcloud config set project <프로젝트 ID>` 로 프로젝트만 정하면 이 교안의 명령을 그대로 쓸 수 있습니다. 다만 Cloud Shell은 로컬 환경과 다른 컴퓨터라서, 실습 저장소도 그 안에 다시 내려받아야 합니다.

객관식 퀴즈

Q. 프로젝트 이름을 `friend-agent-course` 로 만들었는데, 프로젝트 ID는 `friend-agent-course-471203` 으로 정해졌습니다. gcloud 명령의 `--project` 나 `gcloud config set project` 에 넣어야 하는 값은 무엇인가요?

Chapter04

배포 명령 하나가 하는 일

오늘 쓸 배포 명령 `gcloud run deploy --source .` 는 코드를 받아 이미지를 만들고, 실행하고, 주소를 붙이는 일까지 모두 처리합니다. 아래 그림이 그 다섯 단계입니다.



그림: 배포 명령을 실행하면 코드가 빌드, 이미지, 리비전, 인스턴스를 거쳐 서비스 URL이 된다

빌드는 Cloud Build가 맡습니다. 폴더 안의 파일을 보고 Python 앱이라는 것을 알아낸 뒤 Python과 라이브러리를 설치해 이미지로 묶는데, 이렇게 무엇을 설치할지 자동으로 정하는 도구를 buildpacks라고 부릅니다. 만들어진 이미지는 Artifact Registry라는 저장소에 보관됩니다. 리비전은 배포할 때마다 남는 버전으로, 어떤 이미지를 쓸지와 함께 환경변수, 요청 타임아웃, 인스턴스 수 같은 설정이 기록됩니다. 인스턴스는 리비전의 이미지와 설정으로 지금 실행되면서 요청을 처리하는 컨테이너입니다.

그림 아래쪽은 인스턴스 수가 바뀌는 규칙입니다. 한동안 요청이 없으면 인스턴스는 0개가 되고, 그때 들어온 첫 요청은 인스턴스가 켜질 때까지 기다립니다. 꺼져 있던 인스턴스를 새로 켜느라 첫 응답이 늦어지는 것을 콜드 스타트(cold start)라고 합니다. 인스턴스 하나의 동시성은 기본 80이라 요청 80개를 동시에 처리하고, 그보다 많아지면 인스턴스가 늘어나 아무 설정이 없으면 100개까지 늘 수 있습니다. AWS의 ECS Express Mode와 Fargate, Azure의 Container Apps, Cloudflare의 Containers도 같은 구성이라, 이 단계는 다른 클라우드에서도 그대로 찾아볼 수 있습니다.

실습 저장소 내려받기

실습은 템플릿 저장소로 내 저장소를 만들어 시작합니다. 템플릿 저장소 화면 오른쪽 위의 Use this template > Create a new repository를 누르고, 저장소 이름을 정한 뒤 공개

(Public)로 만듭니다. 만들어진 내 저장소를 로컬 환경에 내려받고 그 폴더로 들어갑니다. 섹션 3에서 설정한 gcloud 프로젝트와 리전이 그대로인지도 한 번 더 확인합니다. 섹션 3을 건너뛰고 Cloud Shell을 쓰는 경우에는 Cloud Shell 터미널에서 같은 명령을 실행합니다.

직접 실행할 명령: 저장소 내려받기와 설정 확인

```
# gh 를 쓰는 경우
gh repo clone <내 GitHub 계정>/<저장소 이름>
# git 만 쓰는 경우
git clone https://github.com/<내 GitHub 계정>/<저장소 이름>.git

cd <저장소 이름>
gcloud config list
```

코딩 에이전트에게 맡길 때: 설정 확인

```
지금 폴더가 실습 저장소인지 확인하고, gcloud에 설정된 계정, 프로젝트, Cloud Run 기본 리전을 알려
줘.
확인만 하고 아무것도 바꾸지 마.
```

실행 전 판단 기준

템플릿 저장소에는 작업 규칙이 두 종류로 들어 있습니다. 코딩 에이전트에게 주는 프로젝트 규칙 파일(Claude Code에서는 `CLAUDE.md`)과, 에이전트가 실행할 수 있는 명령을 정하는 권한 설정(Claude Code에서는 `.claude/settings.json`)입니다. 다른 에이전트를 쓴다면 그 도구의 설정에 같은 내용을 옮기고, 손으로 명령을 친다면 두 파일이 그대로 실행 전 점검 목록이 됩니다. 두 파일을 한 번 열어 읽어 두세요. 아래 그림은 그 규칙을 판단 기준으로 정리한 것입니다.



그림: 실행 전 판단 기준. 실행하지 않을 명령, 대상을 확인하고 판단할 명령, 실행할 명령

왼쪽 묶음은 문제를 빨리 해결해 주기 때문에 검색 결과나 에이전트가 지름길로 내놓기 쉬운 명령입니다. 키 파일은 유출되면 누구든 그 계정으로 행동할 수 있고, Editor는 오류와 상관없는 권한까지 함께 줍니다. 권한 오류가 나면 오류 메시지에 나온 작업에 필요한 역할 하나만 줍니다. 작업에 꼭 필요한 권한만 주는 원칙을 최소 권한 원칙(principle of least privilege)이라고 합니다. 가운데 묶음은 대상에 따라 판단이 갈립니다. 곧 쓸 `--allow-unauthenticated` 는 로그인 없이 누구나 호출할 수 있게 하는 인증 없는 호출 허용(unauthenticated invocation) 설정이라, LLM API를 호출하지 않는 연습용 앱에는 문제가 없지만 요청마다 LLM API 비용이 드는 에이전트 서비스라면 인스턴스 수와 LLM API 사용 한도 같은 제한을 함께 걸어야 합니다. 그 설정은 다음 실습에서 다룹니다.

hello 앱 직접 만들기

처음부터 에이전트 서비스를 올리면 실패했을 때 원인이 배포 과정인지 코드인지 가리기 어렵습니다. 그래서 화면 한 장과 헬스 체크 엔드포인트 `/health` 만 있는 연습용 FastAPI 앱을 직접 만들어 먼저 올립니다. 헬스 체크 엔드포인트(health check endpoint)는 서비스가 살아 있는지 알려 주는 주소입니다. 실습 저장소 폴더 밖, 예를 들어 저장소 폴더와 같은 위치에 `my-hello` 폴더를 만들고 아래 파일 네 개를 넣습니다. 손으로 만들어도 되고, 아래 요청 예시로 코딩 에이전트에게 맡겨도 됩니다.

파일	하는 일
`main.py`	앱 코드. `/` 에서 화면 한 장을, `/health` 에서 `{"status":"ok"}` 를 돌려준다
`requirements.txt`	설치할 라이브러리 목록. buildpacks가 이 파일을 보고 의존성을 설치한다

파일	하는 일
`Procfile`	시작 명령. 컨테이너가 켜질 때 이 명령으로 앱을 실행한다
`.python-version`	쓸 Python 버전. buildpacks가 이 버전의 Python을 준비한다

buildpacks는 소스 코드의 파일을 보고 무엇을 설치할지 정해 컨테이너 이미지를 만드는 도구입니다. 그래서 이 세 설정 파일의 이름과 위치가 틀리면 빌드가 실패하거나, 빌드는 되어도 앱이 시작되지 않습니다. `.python-version` 은 2026년 10월 현재 Cloud Run 소스 배포의 기본 빌드 환경이 지원하는 3.13 이상으로 적습니다.

`my-hello/main.py`

```
from fastapiimport FastAPI
from fastapi.responsesimport HTMLResponse

app= FastAPI()


@app.get("/", response_class=HTMLResponse)
defhome():
return"<h1>안녕하세요, 첫 배포입니다</h1>"


@app.get("/health")
defhealth():
return{"status":"ok"}
```

`my-hello/requirements.txt`

```
fastapi>=0.115
uvicorn>=0.30
```

`my-hello/Procfile`

```
web: uvicorn main:app --host 0.0.0.0 --port $PORT
```

`my-hello/.python-version`

```
3.13
```

코딩 에이전트에게 맡길 때: hello 앱 만들기

```
실습 저장소 폴더와 같은 위치에 my-hello 폴더를 만들고, Cloud Run에 소스 배포할 연습용 FastAPI 앱을 만들어 줘.
```

```
- main.py: / 에서 "안녕하세요, 첫 배포입니다"라는 HTML 한 줄, /health 에서 {"status": "ok"}
를 돌려준다
- requirements.txt: fastapi, uvicorn
- Procfile: uvicorn 으로 main:app 을 0.0.0.0 과 PORT 환경변수의 포트에서 실행
- .python-version: 3.13
파일 네 개만 만들고, 다 만들면 각 파일이 하는 일을 한 줄씩 설명해 줘.
```

`Procfile` 의 `$PORT` 는 Cloud Run이 실행할 때 넣어 주는 포트 번호(기본 8080)를 쓰겠다는 뜻이며, 앱이 그 포트를 리슨(listen, 포트를 열고 요청이 들어오기를 기다리는 것)하고 있어야 요청이 전달됩니다. 에이전트가 만든 파일에서 포트를 `8080` 처럼 숫자로 고정했거나, 파일이 네 개보다 많다면 위 내용과 같게 고쳐 달라고 합니다.

hello 앱 배포

파일이 준비되면 `my-hello` 폴더로 들어가 배포합니다. 서비스 이름은 `hello` 로 합니다.

직접 실행할 명령: hello 앱 배포

```
# my-hello/ 폴더 안에서 실행한다
# --source: 현재 폴더의 코드를 올려 빌드  --region: 실행 위치  --allow-unauthenticated: 로그
인 없이 누구나 접속
gcloud run deploy hello --source . --region us-central1 --allow-unauthenticated
```

코딩 에이전트에게 맡길 때: hello 앱 배포

```
my-hello/ 폴더의 앱을 Cloud Run에 소스 배포해 줘.
서비스 이름은 hello, 리전은 us-central1, 로그인 없이 누구나 접속할 수 있게 해 줘.
명령을 실행하기 전에 그 명령이 무엇을 하는지 한 줄씩 설명해 줘.
```

에이전트가 제안한 명령이 위 명령과 같은 이름, 리전, 공개 설정인지 비교하고 실행합니다. 처음 배포하는 프로젝트라면 이미지를 보관할 저장소(`cloud-run-source-deploy`)를 만들겠느냐는 질문이 나오는데, 그림의 3번 단계에서 이미지가 저장되는 곳이므로 `Y` 로 답합니다. 몇 분 뒤 `Service URL` 줄에 서비스 URL(배포하면 Cloud Run이 붙여 주는 공개 주소)이 나오면 브라우저로 열고, 끝에 `/health` 를 붙여 `{"status":"ok"}` 가 보이는지 확인합니다.

콘솔에서 다섯 단계 찾기

터미널에는 결과만 보였지만, 그 사이에 그림의 다섯 단계가 모두 일어났습니다. 각 단계가 남긴 기록을 아래 순서대로 콘솔에서 찾아봅니다. 섹션 3의 바로가기 주소를 쓰면 1, 2번 화면을 바로 열 수 있습니다.

직접 확인할 곳: 다섯 단계의 기록

1. 빌드: 검색창에 `Cloud Build` 를 입력해 Cloud Build를 열고, 왼쪽 메뉴의 기록(History)을 누릅니다. 목록 맨 위 줄이 방금 배포로 생긴 빌드입니다. 목록이 비어 있으면 목록 위쪽의 리전(Region) 선택을 `us-central1` 로 바꿉니다. 빌드 ID(파란 글자)를 누르면 빌드 로그가 열립니다.
2. 컨테이너 이미지: 검색창에 `Artifact Registry` 를 입력해 열고, 저장소 목록에서 `cloud-run-source-deploy` 를 누른 뒤 그 안의 `hello` 를 누릅니다. 배포할 때마다 이미지가 한 줄씩 쌓입니다.
3. 리비전: 검색창에 `Cloud Run` 을 입력해 서비스 목록을 열고, `hello` 를 누른 뒤 위쪽의 리비전(Revisions) 탭을 누릅니다. `hello-00001-xxx` 한 줄과 트래픽 100%가 보입니다.
4. 인스턴스: 같은 서비스 상세 화면에서 측정항목(Metrics) 탭을 누르고, 컨테이너 인스턴스 수(Container instance count) 그래프를 찾습니다. 그래프 위쪽의 기간을 1시간으로 줄이면 방금 켜진 인스턴스가 잘 보입니다.
5. 요청 로그: 같은 화면에서 로그(Logs) 탭을 누릅니다. 브라우저로 연 요청이 `GET 200 /health` 같은 줄로 쌓여 있습니다.

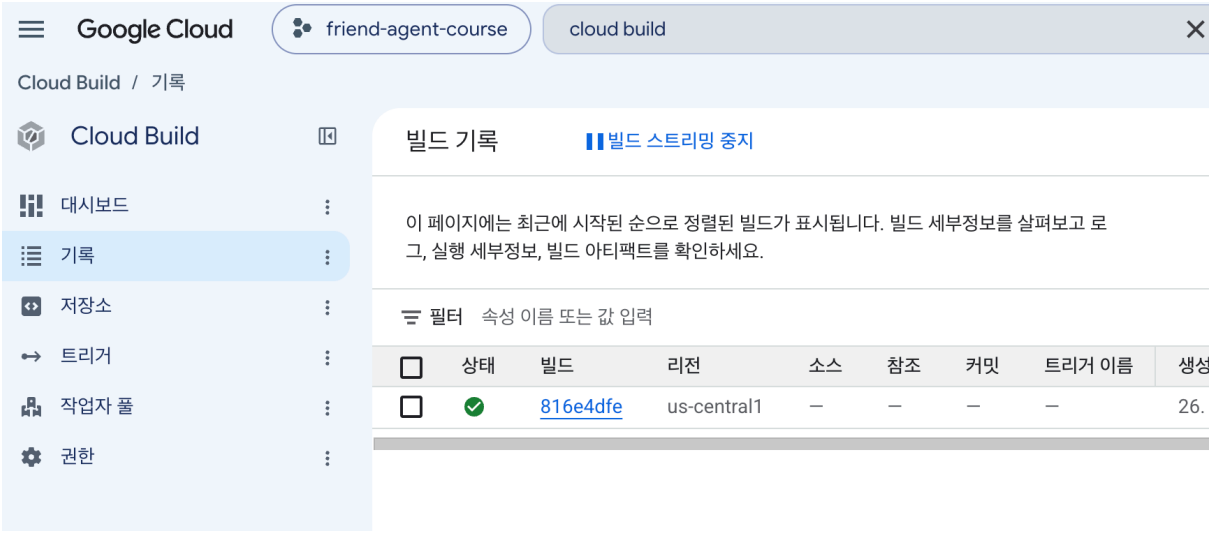


그림: Cloud Build > 기록: hello 배포로 생긴 빌드 한 줄이 성공 표시와 함께 보이는 모습

마지막으로 로그 탭을 열어 방금 브라우저로 연 `/health` 요청 줄을 찾아보세요. 서비스 URL은 누구나 열 수 있어서 모르는 곳에서 온 요청도 여기에 함께 쌓이고, 서비스에 문제가 생기면 가장 먼저 여는 곳도 이 화면입니다.

주관식 퀴즈

Q. [이해 점검] hello 앱을 배포했는데 서비스 URL을 열어도 화면이 나오지 않는다고 가정해 보세요. 콘솔의 어느 화면을 어떤 순서로 열어 원인을

찾을지 이 섹션의 확인 순서를 근거로 적어 보세요.

Chapter05

같은 일을 이번에는 혼자

섹션 4에서는 명령과 요청 예시를 따라 배포했습니다. 이번에는 같은 흐름을 스스로 한 번 더 해 봅니다. 아래 순서를 먼저 직접 해 보고, 막히는 단계에서만 섹션 4의 명령이나 이 섹션의 요청 예시를 엽니다. 따라 할 때는 쉬워 보였던 단계도 혼자 하면 어디서 확인해야 할지 망설이게 되는데, 그 망설인 자리가 다음에 다시 볼 곳입니다.

1. `my-hello/main.py` 의 화면 문구를 "두 번째 배포입니다"로 바꿉니다.
2. 같은 배포 명령으로 다시 배포하고, 브라우저에서 문구가 바뀌었는지 봅니다.
3. 리비전 목록을 보고 리비전이 몇 개인지, 지금 트래픽을 받는 리비전이 어느 것인지 적습니다.
4. 트래픽 100%를 첫 리비전으로 전환해 화면이 처음 문구로 돌아가는지 봅니다.
5. 다시 최신 리비전이 트래픽을 받게 돌려놓습니다.

직접 실행할 명령: 다시 배포하고 리비전 목록 보기

```
# my-hello/ 폴더 안에서, 화면 문구를 바꾼 뒤
gcloud run deploy hello --source . --region us-central1 --allow-unauthenticated

# 리비전 목록: 이름, 트래픽 비율, 만든 시각
gcloud run revisions list --service hello --region us-central1
```

직접 확인할 곳: 리비전 두 개 확인

1. 검색창에 `Cloud Run` 을 입력해 서비스 목록을 열고 `hello` 를 누릅니다.
2. 위쪽의 리비전(Revisions) 탭을 누릅니다. 리비전이 한 줄에 하나씩, 최신 리비전이 맨 위에 보입니다.
3. 각 줄의 트래픽(Traffic) 칸에서 어느 리비전이 트래픽을 받는지 봅니다. 화면이 바로 바뀌지 않으면 위쪽의 새로고침 버튼을 누릅니다.

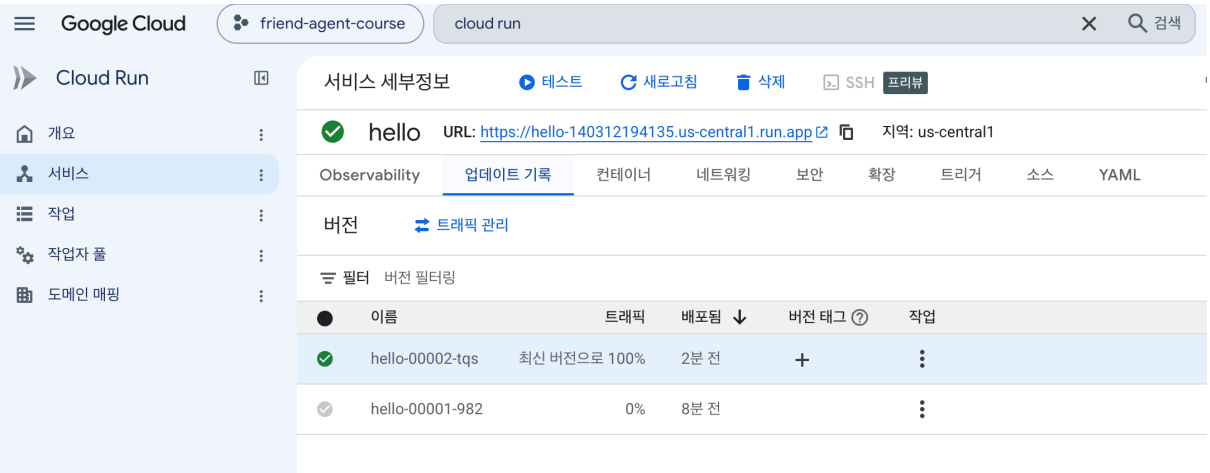


그림: Cloud Run > hello > 리비전 탭: hello-00002 리비전이 트래픽 100%, hello-00001 리비전이 0%로 나란히 보이는 모습

새 리비전 `hello-00002-` 가 트래픽 100%를 받고, 첫 리비전은 지워지지 않고 0%로 남아 있습니다. 서비스 URL은 그대로입니다. 리비전은 배포할 때마다 남는 버전이라서, 새 버전에 문제가 생기면 코드를 고쳐 다시 빌드하지 않고도 트래픽을 이전 리비전으로 전환할 수 있습니다. 문제가 생긴 새 버전 대신 이전 버전으로 되돌리는 것을 롤백(rollback)이라고 하며, 4번 단계가 롤백입니다.

트래픽을 전환하는 명령은 `update-traffic` 입니다. `--to-revisions` 에는 리비전 이름과 비율을 적고, `--to-latest` 는 "앞으로 가장 최근 리비전이 트래픽을 받는다"는 뜻입니다. 리비전 이름은 3번에서 적은 목록에서 그대로 복사합니다.

직접 실행할 명령: 이전 리비전으로 롤백했다가 최신으로 돌려놓기

```
# 4. 트래픽 100%를 첫 리비전으로 (이름은 리비전 목록에서 복사한다)
gcloud run services update-traffic hello --region us-central1 --to-revisions <첫 리비전 이름>=100

# 5. 다시 최신 리비전이 트래픽을 받게
gcloud run services update-traffic hello --region us-central1 --to-latest
```

코딩 에이전트에게 맡길 때: 막혔을 때

```
hello 서비스의 리비전 목록을 보여 주고, 지금 트래픽을 받는 리비전이 어느 것인지 알려 줘.
그다음 트래픽 100%를 첫 번째 리비전으로 전환하는 명령을 실행 전에 먼저 보여 줘. 내가 확인하면 실행해 줘.
```

에이전트가 보여 준 명령의 리비전 이름이 목록의 첫 리비전과 같은지, 서비스 이름이 `hello` 인지 확인하고 승인합니다. 4번을 한 뒤 브라우저를 새로고침하면 처음 문구가 보이고, 리비

전 탭에서는 트래픽 비율이 바뀐 것을 볼 수 있습니다. 트래픽을 특정 리비전에 고정해 두면 다음에 배포한 새 리비전이 트래픽을 받지 않으므로, 5번으로 꼭 돌려놓습니다.

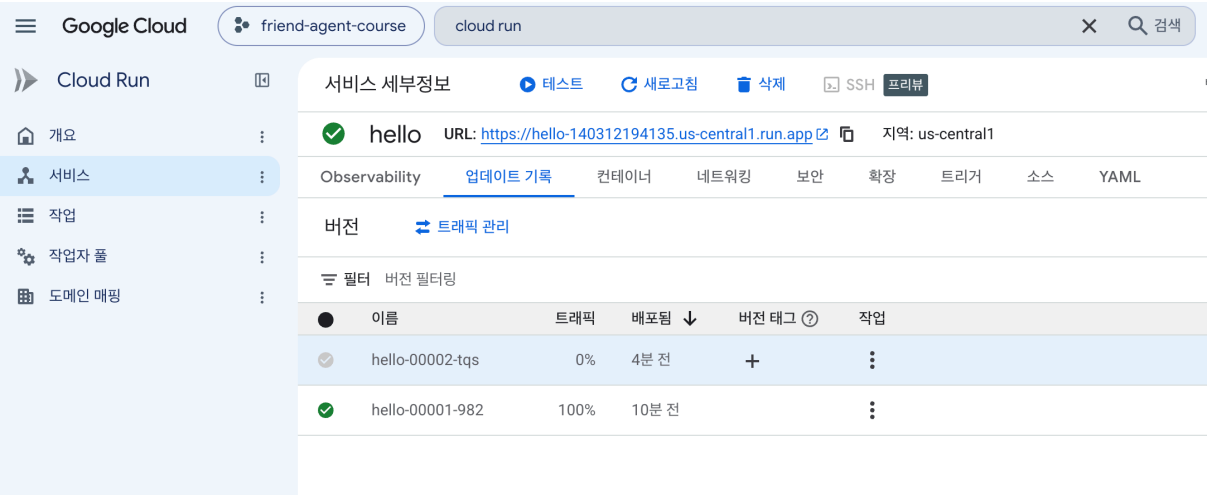


그림: Cloud Run > hello > 리비전 탭 (4번을 실행한 뒤): hello-00001 리비전이 트래픽 100%, hello-00002 리비전이 0%로 바뀐 모습

더 해 보기: 트래픽을 반씩 나누기

트래픽은 한 리비전에 모두 보낼 필요가 없고, 리비전 사이에 비율을 나눠 보내는 것을 트래픽 분할(traffic splitting)이라고 합니다. 두 리비전에 50%씩 나누고 서비스 URL을 여러 번 새로고침해 보세요. 새로고침할 때마다 Cloud Run이 요청을 어느 리비전으로 보낼지 따로 정하므로, 처음 문구와 "두 번째 배포입니다"가 섞여 나옵니다. 열 번쯤 새로고침해서 각 문구가 몇 번 나왔는지 세어 봅니다. 정확히 다섯 번씩 나오지 않아도 정상이고, 횟수를 늘릴수록 반반에 가까워집니다. 브라우저가 같은 화면을 계속 보여 주면 시크릿 창에서 열거나 아래 명령으로 여러 번 요청합니다.

새 버전을 일부 사용자에게만 먼저 보여 주고 문제가 없으면 비율을 늘리는 카나리 배포(canary deployment)도 트래픽 분할로 만듭니다. 확인했으면 `--to-latest` 로 돌려놓습니다.

직접 실행할 명령: 트래픽을 반씩 나누고 여러 번 요청하기

```
# 두 리비전에 50%씩 (이름은 리비전 목록에서 복사한다)
gcloud run services update-traffic hello --region us-central1 --to-revisions <첫 리비전 이름>=50,<둘째 리비전 이름>=50

# 열 번 요청해서 문구 세기 (macOS 터미널)
for i in $(seq 1 10); do curl -s <공개 주소>; echo; done
# Windows PowerShell
1..10 | ForEach-Object { (Invoke-WebRequest -UseBasicParsing <공개 주소>).Content }
```

```
# 확인한 뒤 최신 리비전으로 돌려놓기
gcloud run services update-traffic hello --region us-central1 --to-latest
```

코딩 에이전트에게 맡길 때: 트래픽을 반씩 나누기

```
hello 서비스의 리비전 두 개에 트래픽을 50%씩 나눠 줘. 실행하기 전에 두 리비전 이름과 비율을 먼저 보여 줘.
끝나면 서비스 URL에 열 번 요청해서 각 문구가 몇 번 나왔는지 세어 줘.
```

리비전 탭의 트래픽 칸이 50%, 50%로 바뀌었는지 콘솔에서도 확인합니다. 에이전트가 센 횟수와 내가 브라우저에서 본 결과를 비교해 보세요.

객관식 퀴즈

Q. 에이전트 서비스를 한 번 배포한 뒤, 코드는 그대로 두고 요청 타임아웃 설정만 바꿨습니다. Cloud Run에서 무엇이 달라질까요?

- 1. 아무것도 달라지지 않는다. 코드가 같으므로 리비전도 그대로다
- 2. 새 리비전이 하나 생긴다. 리비전에는 이미지뿐 아니라 설정도 함께 기록되기 때문이다
- 3. 새 서비스 URL이 하나 더 생긴다
- 4. 인스턴스가 항상 켜진 상태로 바뀐다

- •

  1. 아무것도 달라지지 않는다. 코드가 같으므로 리비전도 그대로다

- •

  2. 새 리비전이 하나 생긴다. 리비전에는 이미지뿐 아니라 설정도 함께 기록되기 때문이다

- •

  3. 새 서비스 URL이 하나 더 생긴다

- •

  4. 인스턴스가 항상 켜진 상태로 바뀐다

Chapter06

배포 전 로컬 실행

`hello` 앱은 배포만 해 보면 되는 연습용이었지만, 에이전트 서비스는 내가 만들어 사용자에게 내놓는 서비스입니다. 배포하기 전에 로컬 환경(내 컴퓨터)에서 한 번 실행해 보면, 나중에 서버에서 문제가 생겼을 때 코드 문제인지 배포 환경 문제인지 가르기 쉽습니다. 로컬 환경에서 잘 돌던 것이 서버에서만 안 되면 둘의 차이를 찾으면 되기 때문입니다.

에이전트 서비스는 시작할 때 환경변수에서 LLM API 키(`LLM_API_KEY`)와 LLM 제공사(`LLM_PROVIDER`)를 읽습니다. 로컬 환경에서는 이 값을 저장소 루트의 `.env` 파일에 적어 두면 앱이 시작할 때 읽어 들입니다. 템플릿에는 빈칸만 있는 `.env.example` 이 들어 있으니, 이 파일을 `.env` 로 복사해 키를 채웁니다. 이 과정에서 쓰는 키는 강사가 나눠 준 회사 키입니다.

직접 실행할 명령: 설치하고 .env 만들기

```
# 저장소 루트에서 실행한다
python -m venv .venv
source .venv/bin/activate            # Windows: .venv\Scripts\activate
pip install -r requirements.txt

cp .env.example .env                 # Windows: copy .env.example .env
```

코딩 에이전트에게 맡길 때: 설치하고 .env 만들기

```
저장소 루트에 가상환경을 만들고 requirements.txt 를 설치해 줘.
그다음 .env.example 을 .env 로 복사만 해 줘. 키는 내가 직접 넣을 거니까 .env 내용은 읽거나 출력하지 마.
```

키 값은 편집기로 `.env` 를 직접 열어 넣습니다. 에이전트 대화창에 붙여 넣으면 키가 대화 기록에 남기 때문입니다. `LLM_PROVIDER=fake` 를 `openai` 로 바꾸고(받은 키가 Gemini 키라면 `gemini`), `LLM_API_KEY=` 뒤에 키를 붙여 넣은 뒤 저장합니다.

직접 실행할 명령: 로컬 환경에서 실행

```
uvicorn app.main:app --port 8080
# 브라우저에서 http://localhost:8080 을 연다. 끝낼 때는 터미널에서 Ctrl+C
```

브라우저에서 `http://localhost:8080` 을 열면 캐릭터 만들기 화면이 나옵니다. 캐릭터를 저장하고 몇 마디 걸어 답이 오는지 봅니다. 오른쪽 "이번 턴" 패널에는 고른 도구와 걸린 시간이 나오는데, 로컬 환경에서 잰 이 시간이 다음 실습에서 요청 타임아웃을 정할 때 비교 기준이 됩니다. 앱이 바로 꺼지고 터미널에 `LLM_API_KEY` 가 없다는 줄이 나오면 `.env` 를 저장했는지, 키 앞뒤에 따옴표나 공백이 들어가지 않았는지 확인합니다.

.env 는 어디까지 가는가

실행이 끝났으면 터미널에서 `git status` 를 실행해 보세요. 방금 만든 `.env` 가 목록에 나오지 않습니다. 템플릿의 `.gitignore` 에 `.env` 가 적혀 있어 Git이 이 파일을 무시하기 때문입니다. 소스 배포도 같습니다. `gcloud run deploy --source .` 는 폴더를 통째로 올리기 전에 `.gcloudignore` 에 적힌 파일을 빼는데, 여기에도 `.env` 가 들어 있습니다. 그래서 `.env` 의 키는 로컬 환경에만 있고, 저장소에도 Cloud Run에도 가지 않습니다.

직접 실행할 명령: .env 가 빠지는지 확인

```
git status                          # .env 가 보이지 않아야 한다
grep -n "^.env$" .gitignore .gcloudignore
# Windows PowerShell: Select-String -Pattern "^.env$" .gitignore, .gcloudignore
```

객관식 퀴즈

Q. 로컬 환경의 `.env` 에 키를 넣고 앱이 잘 도는 것을 확인했습니다. 같은 폴더에서 `gcloud run deploy --source .` 로 배포하면 서버의 앱은 키를 어떻게 얻을까요?

• 1. `.env` 가 소스와 함께 올라가므로 서버에서도 그대로 읽는다
• 2. `.gcloudignore` 가 `.env` 를 빼고 올리므로 서버에는 키가 없다. 서버에 키를 따로 전달해야 한다
• 3. Cloud Run이 로컬 환경의 `.env` 를 배포할 때마다 자동으로 복사해 간다
• 4. `git status` 에 `.env` 가 보이지 않으므로 키는 GitHub에 저장된다

- •

  1. `.env` 가 소스와 함께 올라가므로 서버에서도 그대로 읽는다

- •

  2. `.gcloudignore` 가 `.env` 를 빼고 올리므로 서버에는 키가 없다. 서버에 키를 따로 전달해야 한다

- •

  3. Cloud Run이 로컬 환경의 `.env` 를 배포할 때마다 자동으로 복사해 간다

- •

  4. `git status` 에 `.env` 가 보이지 않으므로 키는 GitHub에 저장된다

Chapter07

키 없이 올라간 첫 배포

섹션 6에서 확인한 대로 `.env` 는 배포에 올라가지 않습니다. 그 상태로 배포하면 어떻게 되는지 먼저 봅니다. `hello` 앱과 같은 명령에서 서비스 이름만 `friend-agent` 로 바꿉니다.

직접 실행할 명령: 에이전트 서비스 배포

```
# 저장소 루트에서 실행한다
gcloud run deploy friend-agent --source . --region us-central1 --allow-unauthenticated
```

코딩 에이전트에게 맡길 때: 에이전트 서비스 배포

```
저장소 루트의 에이전트 서비스를 Cloud Run에 소스 배포해 줘.
서비스 이름은 friend-agent, 리전은 us-central1, 로그인 없이 누구나 접속할 수 있게 해 줘.
키는 아직 연결하지 말고 이대로 배포해서 결과를 보여 줘.
```

빌드는 성공하지만 마지막 단계에서 실패하고, 터미널에는 "컨테이너가 PORT=8080에서 시작하지 못했다"는 내용의 메시지가 나옵니다. 포트 문제처럼 보이지만 같은 구조의 `hello` 앱은 문제없이 올라갔습니다. 아래 그림은 이때 Cloud Run 안에서 일어난 일입니다.



그림: LLM API 키 없이 배포했을 때의 순서. 터미널 메시지는 결과를, 로그는 원인을 보여준다

에이전트 서비스는 시작하자마자 LLM API 키를 확인하고, 키가 없으면 오류 로그를 남긴 뒤 바로 종료합니다. 앱이 꺼졌으니 포트를 리슨(listen, 포트를 열고 요청이 들어오기를 기다리는 것)하는 프로그램도 없고, Cloud Run은 "포트에서 요청을 받을 준비가 되지 않았다"고 판단합니다. 포트 문제는 결과이고 원인은 앞서 앱이 스스로 꺼진 데 있습니다. 키가 없을 때

일부러 꺼지게 만든 것은, 그래야 사용자에게 오류를 돌려주는 서비스가 열리기 전에 배포 단계에서 바로 실패하고 트래픽이 새 리비전으로 전환되지 않기 때문입니다.

원인은 로그에서 찾습니다. 에이전트에게 원인을 물을 때는 "로그를 근거로"와 "아직 고치지 마"를 함께 넣습니다. 그러지 않으면 터미널 메시지만 보고 멀쩡한 포트 설정을 고치자고 제안할 수 있습니다.

직접 확인할 곳: 실패한 리비전의 로그

1. 검색창에 `Cloud Run` 을 입력해 서비스 목록을 엽니다. 배포에 실패해도 `friend-agent` 는 목록에 있고, 이름 옆에 오류 표시가 붙어 있습니다.
2. `friend-agent` 를 누르고 위쪽의 로그(Logs) 탭을 누릅니다.
3. 로그 목록 위쪽의 심각도(Severity) 선택을 눌러 오류(Error)를 고릅니다. 줄이 너무 많으면 기간을 지난 1시간으로 줄입니다.
4. `LLM_API_KEY` 가 없어 앱을 시작하지 않는다는 줄을 찾고, 줄 앞의 화살표를 눌러 펼쳐 봅니다.

코딩 에이전트에게 맡길 때: 실패 원인 찾기

```
friend-agent 배포가 실패했어. 실패한 리비전의 Cloud Run 로그를 읽고, 로그에 적힌 내용을 근거로 원인을 설명해 줘.
아직 아무것도 고치지 마.
```

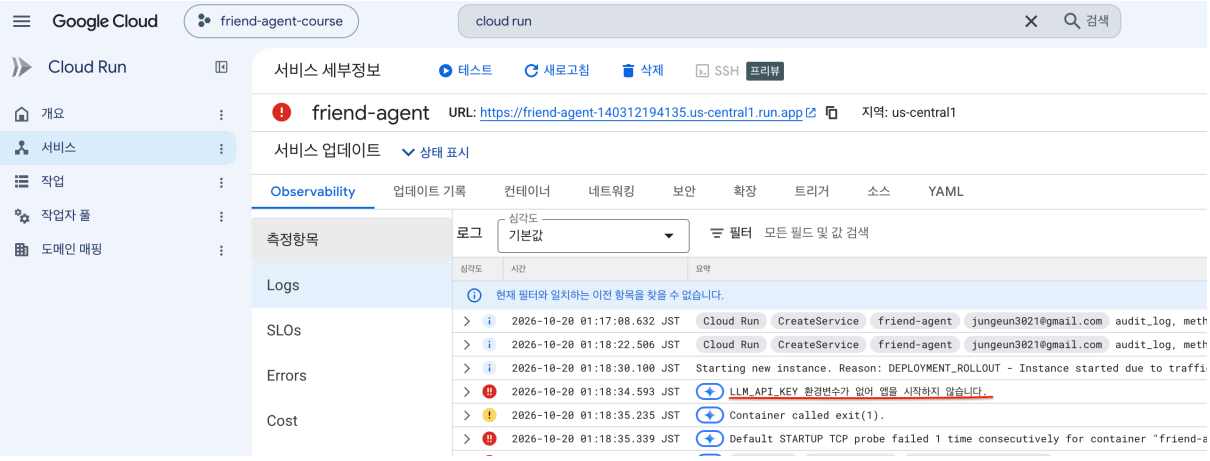


그림: Cloud Run > friend-agent > 로그 탭, 심각도 필터 오류: LLM_API_KEY 환경변수가 없어 앱을 시작하지 않는다는 오류 줄

서버에 키를 전달하는 방법

서버의 앱에도 `LLM_API_KEY` 환경변수가 있어야 합니다. Cloud Run에 환경변수를 넣는 방법은 두 가지입니다. 하나는 배포 명령에 키 값을 그대로 적어 일반 환경변수로 넣는 것이고, 다른 하나는 키를 Secret Manager에 따로 보관하고 서비스에는 시크릿의 이름만 적는 것입니다. 아래 그림은 두 방법에서 키 값이 어디에 남는지 비교한 것입니다.

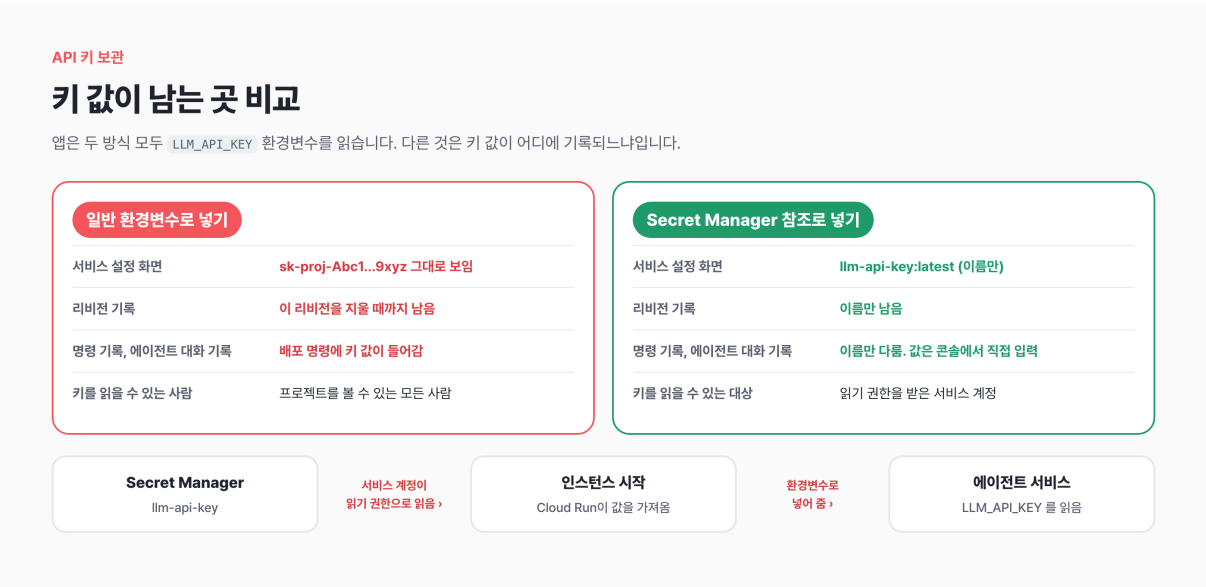


그림: 일반 환경변수와 Secret Manager 참조에서 키 값이 남는 곳 비교

일반 환경변수로 넣은 값은 리비전 설정의 일부로 기록됩니다. 프로젝트를 볼 수 있는 사람은 콘솔의 리비전 상세 화면에서 키를 그대로 읽을 수 있고, 키를 바꿔도 이전 리비전마다 예전 키가 남습니다. 배포 명령에 키를 적었으니 터미널 기록과 에이전트 대화 기록에도 키가 남습니다.

Secret Manager에 두면 키 값은 한 곳에만 있고, 서비스 설정에는 `llm-api-key` 같은 이름만 기록됩니다. 인스턴스가 시작될 때 Cloud Run이 그 이름으로 값을 읽어 환경변수로 넣어 주므로 앱 코드는 로컬 환경에서와 똑같이 `LLM_API_KEY` 를 읽습니다. 누가 이 값을 읽을 수 있는지는 시크릿마다 권한으로 정하고, 키를 새 값으로 바꾸는 키 교체(키 로테이션, key rotation) 때는 같은 이름에 새 버전을 추가합니다. 템플릿의 작업 규칙에도 시크릿은 Secret Manager에 두고 이름으로 연결한다고 적혀 있어서, 코딩 에이전트에게 키를 환경변수에 그대로 넣어 달라고 하면 Secret Manager를 쓰자고 되묻는 경우가 많습니다. 오늘은 처음부터 Secret Manager로 연결합니다.

키 값을 넣는 일은 콘솔에서 직접 합니다. 명령으로 넣거나 에이전트에게 맡기면 키 값이 다시 명령 기록과 대화 기록에 들어가기 때문입니다.

직접 확인할 곳: 시크릿 만들기

1. 검색창에 `Secret Manager` 를 입력해 엽니다. 섹션 3에서 바로가기 주소를 써도 됩니다.
2. 위쪽의 보안 비밀 만들기(Create secret)를 누릅니다.

3. 이름(Name)에 `llm-api-key` 를 입력합니다.
4. 보안 비밀 값(Secret value) 칸에 `.env` 에 넣었던 것과 같은 키를 붙여 넣습니다. 나머지 항목은 기본값으로 두고 맨 아래 보안 비밀 만들기를 누릅니다.
5. 시크릿 목록에 `llm-api-key` 가 보이고, 이름을 눌렀을 때 버전 1이 사용 설정됨(Enabled) 상태인지 확인합니다.

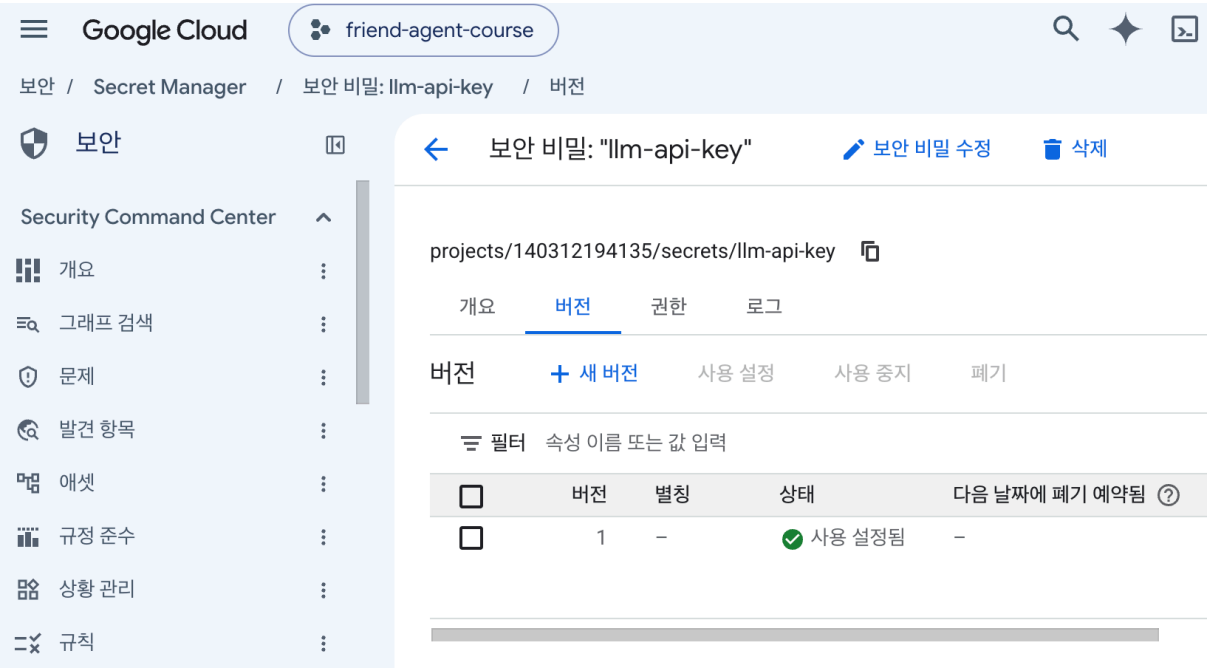


그림: Secret Manager > llm-api-key 상세 화면: 버전 1이 사용 설정됨 상태로 보이는 모습 (값은 보이지 않는다)

다음은 연결입니다. 서비스 계정은 사람 대신 프로그램이 쓰는 Google Cloud 계정이고, 서비스가 실행되는 동안 쓰는 서비스 계정을 런타임 서비스 계정이라고 합니다. 에이전트 서비스는 Cloud Run이 기본으로 쓰는 서비스 계정(`<프로젝트 번호>-compute@developer.gserviceaccount.com`)을 런타임 서비스 계정으로 쓰므로, 이 계정이 `llm-api-key` 하나만 읽을 수 있게 권한을 주고, 서비스의 `LLM_API_KEY` 가 이 시크릿을 참조하게 합니다. 이 단계에서는 시크릿의 이름만 오갑니다. 설정만 바꾸는 일이라 다시 빌드하는 `deploy` 대신 `update` 를 씁니다.

직접 실행할 명령: 권한 주고 시크릿 연결하기

```
# 1. 프로젝트 번호 확인
gcloud projects describe <PROJECT_ID> --format="value(projectNumber)"

# 2. llm-api-key 하나에 대해서만 읽기 권한 주기
gcloud secrets add-iam-policy-binding llm-api-key --member="serviceAccount:<프로젝트 번호>-compute@developer.gserviceaccount.com" --role="roles/secretmanager.secretAccessor"
```

```
# 3. LLM_API_KEY 가 시크릿의 최신 버전을 참조하게 하기
gcloud run services update friend-agent --region us-central1 --update-secrets LLM_API_
KEY=llm-api-key:latest
# 받은 키가 Gemini 키라면 3번 명령 끝에 --update-env-vars LLM_PROVIDER=gemini 를 붙인다
```

코딩 에이전트에게 맡길 때: 권한 주고 시크릿 연결하기

```
Secret Manager에 llm-api-key라는 시크릿을 만들어 두었어. 키 값은 읽거나 출력하지 마.
friend-agent의 런타임 서비스 계정에 llm-api-key 하나에 대해서만 읽기 권한을 주고,
LLM_API_KEY 환경변수가 llm-api-key의 최신 버전을 참조하도록 서비스 설정을 바꿔 줘.
명령마다 무엇을 바꾸는지 먼저 설명해 줘.
```

권한 명령은 역할이 읽기 전용인 `roles/secretmanager.secretAccessor` 인지, 대상이 프로젝트 전체가 아니라 시크릿 하나(`gcloud secrets add-iam-policy-binding llm-api-key`)인지 확인하고 승인합니다. 끝나면 서비스 URL을 브라우저로 열어 캐릭터를 만들고 몇 마디 걸어, 로컬 환경에서처럼 답이 오는지 확인합니다.

직접 확인할 곳: 리비전에 기록된 키 설정

1. Cloud Run > `friend-agent` > 리비전(Revisions) 탭을 엽니다.
2. 맨 위 리비전의 이름을 누르면 오른쪽에 상세 패널이 열립니다.
3. 패널의 컨테이너(Containers) 탭에서 변수 및 보안 비밀(Variables & Secrets) 항목을 찾습니다. `LLM_API_KEY` 옆에 키 값 대신 `llm-api-key` 와 버전 `latest` 가 보이면 됩니다.

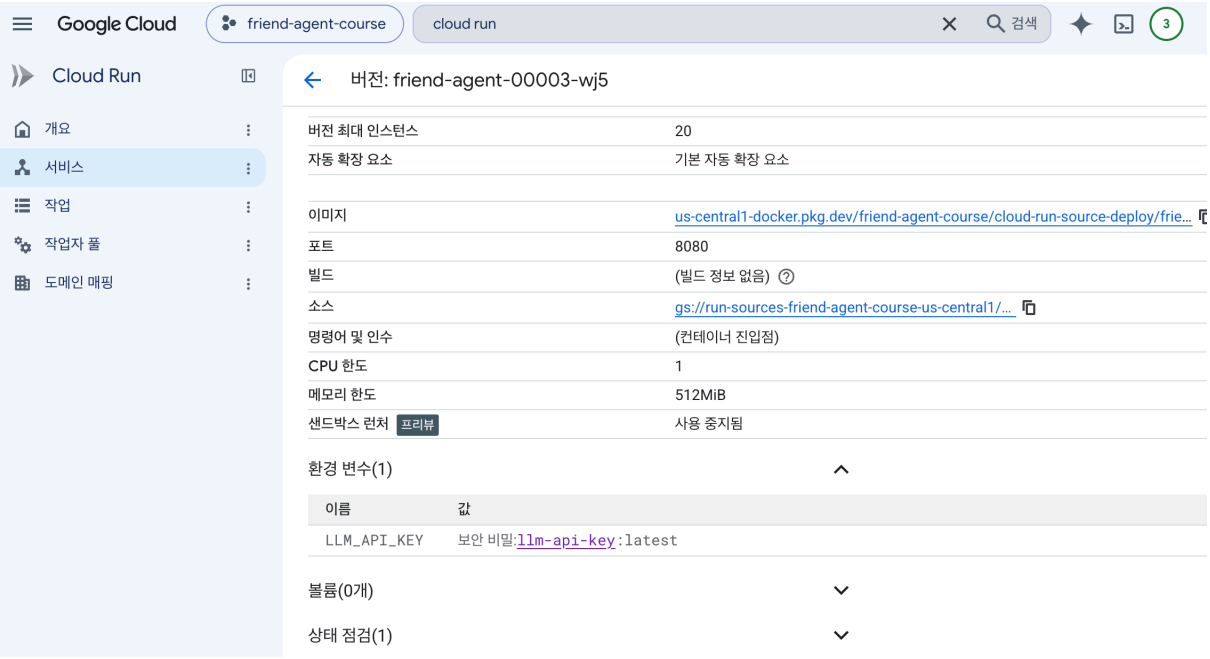


그림: Cloud Run > friend-agent > 리비전 탭 > 맨 위 리비전 > 컨테이너 탭 > 변수 및 보안 비밀: LLM_API_KEY가 llm-api-key의 latest 버전을 참조하고 키 값은 보이지 않는 모습

키 교체와 남은 한계

키를 교체할 때는 Secret Manager의 `llm-api-key` 를 열고 새 버전 추가(New version)로 새 키를 넣습니다. 서비스는 `latest` 를 참조하지만, 이미 실행 중인 인스턴스는 시작할 때 읽은 값을 계속 쓰므로 `gcloud run services update friend-agent --region us-central1 --update-secrets LLM_API_KEY=llm-api-key:latest` 를 다시 실행해 새 리비전을 띄워야 새 키가 적용됩니다. 새 키로 답이 오는 것을 확인한 뒤에 예전 버전을 사용 중지하면, 바꾸는 동안에도 서비스가 멈추지 않습니다.

남은 한계도 하나 있습니다. 오늘 읽기 권한을 준 기본 서비스 계정은 프로젝트의 다른 서비스도 함께 쓰는 계정이라, 같은 계정으로 실행되는 다른 서비스도 이 시크릿을 읽을 수 있습니다.

객관식 퀴즈

Q. 동료가 "Secret Manager는 번거로우니 배포 명령에 `--update-env-vars LLM_API_KEY=<키 값>` 을 쓰자"고 합니다. 이렇게 했을 때 생기는 일로 옳은 것은 무엇인가요?

• 1. 앱이 키를 읽지 못해 시작에 실패한다
• 2. 키 값이 리비전 설정에 그대로 기록되어 프로젝트를 볼 수 있는 사람이 콘솔에서 읽을 수 있고, 키를 바꿔도 이전 리비전에 남는다
• 3. 키 값이 자동으로 암호화되어 콘솔에는 이름만 보인다
• 4. Secret Manager를 쓸 때와 차이가 없다**제출**

- •

  1. 앱이 키를 읽지 못해 시작에 실패한다

- •

  2. 키 값이 리비전 설정에 그대로 기록되어 프로젝트를 볼 수 있는 사람이 콘솔에서 읽을 수 있고, 키를 바꿔도 이전 리비전에 남는다

- •

  3. 키 값이 자동으로 암호화되어 콘솔에는 이름만 보인다

- •

  4. Secret Manager를 쓸 때와 차이가 없다

주관식 퀴즈

Q. [이해 점검] 첫 배포의 터미널 메시지는 포트를 이야기했지만 실제 원인은 LLM API 키였습니다. 메시지만 보고 고쳤다면 어떤 일이 생겼을지, 직접 고칠 때와 에이전트에게 맡길 때 각각 어떻게 하면 막을 수 있는지 설명해 보세요.

0/300**다음**

📚 키 관리 참고 자료

- Configure secrets for services: Cloud Run에서 시크릿을 환경변수로 넣는 방법, 필요한 권한, 버전을 바꿀 때의 동작을 확인할 수 있습니다.
- gcloud run services update: 오늘 쓴 시크릿 연결 옵션과, 다음 실습에서 쓸 요청 타임아웃과 인스턴스 수 옵션을 한곳에서 확인할 수 있습니다.

연습용 서비스 정리

`hello` 서비스는 오늘 연습으로만 썼으니 지웁니다. Cloud Run 서비스를 지워도 배포할 때 만들어진 컨테이너 이미지는 Artifact Registry에 그대로 남습니다. 이미지는 배포할 때마다 한 버전씩 쌓이고, Artifact Registry의 무료 저장 공간은 월 0.5GB라서 남겨 두면 저장 비용이 나갈 수 있습니다. 그래서 서비스와 이미지를 함께 지웁니다.

이때 저장소 `cloud-run-source-deploy` 자체는 지우지 않습니다. 이 저장소에는 `friend-agent`의 이미지도 함께 들어 있어서, 저장소를 통째로 지우면 지금 서비스 중인 에이전트 서비스의 이미지까지 사라집니다. 저장소 안에서 `hello` 패키지만 지웁니다. 삭제는 실행 전에 한 번 더 묻는데, 지우는 대상이 `hello` 이고 리전이 `us-central1` 인지 확인하고 진행합니다.

직접 실행할 명령: hello 서비스와 이미지 삭제

```
# 1. Cloud Run 서비스 삭제
gcloud run services delete hello --region us-central1

# 2. Artifact Registry 에서 hello 이미지(패키지)만 삭제. 저장소 cloud-run-source-deploy 는
남긴다
gcloud artifacts packages delete hello --repository cloud-run-source-deploy --location
us-central1

# 3. 확인: 저장소에 friend-agent 만 남았는지
gcloud artifacts packages list --repository cloud-run-source-deploy --location us-cent
ral1
```

코딩 에이전트에게 맡길 때: hello 서비스와 이미지 삭제

```
us-central1의 hello Cloud Run 서비스를 삭제하고, Artifact Registry 저장소 cloud-run-source
-deploy 안의 hello 패키지만 삭제해 줘.
저장소 자체와 friend-agent 서비스, friend-agent 이미지는 건드리지 마.
삭제 명령을 실행하기 전에 지우는 대상의 이름과 리전을 다시 보여 주고, 끝나면 저장소에 남은 패키지 목
록을 보여 줘.
```

콘솔에서도 할 수 있습니다. Artifact Registry > `cloud-run-source-deploy` 를 열면 패키지 목록에 `hello` 와 `friend-agent` 가 보입니다. `hello` 왼쪽의 체크박스만 선택하고 위쪽의 삭제를 누릅니다. 저장소 목록 화면에서 `cloud-run-source-deploy` 를 선택해 삭제하지 않도록 주의합니다.

Chapter08

오늘 배운 것

Google Cloud. 계정 안에 결제 계정이 있고, 결제 계정에 연결된 프로젝트 안에서 API를 켜고 리전에 서비스를 만듭니다. 명령은 프로젝트 이름이 아니라 프로젝트 ID로 프로젝트를 가리키고, 로컬 환경의 gcloud CLI와 콘솔과 Cloud Shell은 모두 같은 프로젝트를 다룹니다.

배포 구조. 서버리스 컨테이너는 코드와 런타임, 의존성을 묶은 이미지를 받아 실행하고, 배포 명령 하나가 빌드, 이미지 저장, 리비전, 인스턴스, 서비스 URL을 차례로 만듭니다. 이 단계들을 콘솔의 Cloud Build 기록, 리비전 탭, 로그 탭에서 하나씩 찾았고, 트래픽을 이전 리비전으로 롤백하거나 두 리비전에 반씩 나눠 보았습니다.

키가 가는 길. 로컬 환경에서는 `.env` 에서 키를 읽었지만, `.env` 는 `.gitignore` 와 `.gcloudignore` 때문에 저장소에도 배포에도 올라가지 않았습니다. 그래서 키 없이 올라간 첫 배포는 실패했고, 터미널 메시지는 포트를 이야기했지만 원인은 로그에 적힌 키 누락이었습니다. 서버에는 키를 Secret Manager에 두고 이름으로 연결했습니다.

명령을 다루는 방식. 직접 입력하든 에이전트에게 맡기든, 무엇을 어디에 어떤 이름으로 바꿀지 먼저 정하고, 실행 전에 명령이 무엇을 만들고 바꾸고 지우고 공개하는지 읽고, 결과는 콘솔에서 확인했습니다. 키처럼 비밀인 값은 콘솔에서 직접 넣었습니다.

주관식 퀴즈

Q. [회고] 섹션 1에서 Vercel 배포 버튼을 누른 뒤 일어나는 일을 짐작해 적었습니다. 그 답을 다시 읽고, 오늘 콘솔에서 직접 찾은 단계와 비교했을 때 짐작이 맞았던 부분과 새로 알게 된 부분을 적어 보세요.